Overlap-Add and Overlap-Save Method for Filtering Long Data Sequences

Term Assignment Report
Digital Signal Processing - 2EC402CC23

Nirma University
School of Technology, Institute of Technology
Electronics and Communication Engineering Department
Semester – IV (Even)

Sakdecha Rushik - 23BEC170
Shah Sharad - 23BEC180

Contents

1 Problem Statement

Filtering long data sequences using the Overlap-Add and Overlap-Save methods to efficiently compute the convolution of large signals by segmenting the input signal and combining the results.

2 Description

When input signals contain a large number of samples, direct computation of Discrete Fourier Transform (DFT) and Inverse Discrete Fourier Transform (IDFT) becomes computationally expensive. Instead, the signal is divided into smaller blocks, processed separately using convolution with a given filter, and then combined using two primary methods:

- **Overlap-Add Method:** Each block is convolved separately, and results are added while considering overlapping regions.
- **Overlap-Save Method:** Convolution is performed on overlapping segments, but only the non-overlapping portion is retained.

3 Algorithm

3.1 Overlap-Add Method

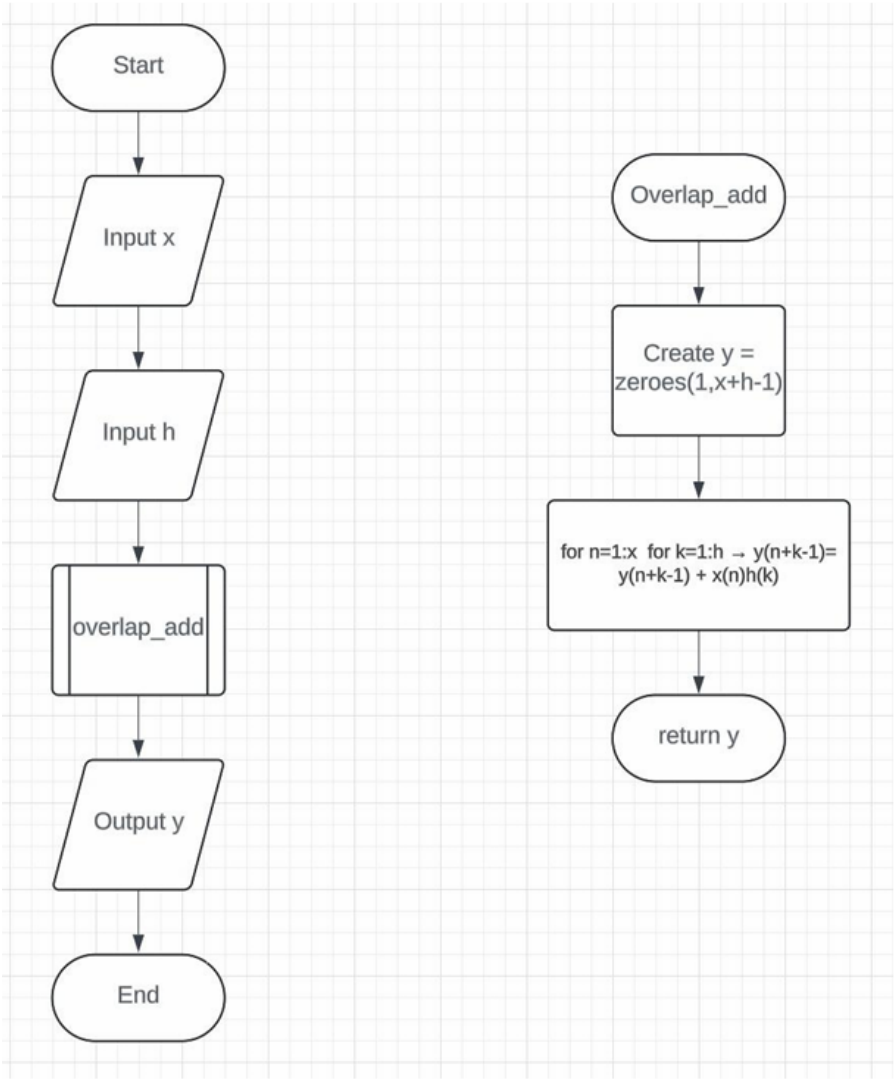
1. Divide the input signal into smaller blocks.
2. Convolve each block separately with the impulse response.
3. Add overlapping segments to reconstruct the final output.

3.2 Overlap-Save Method

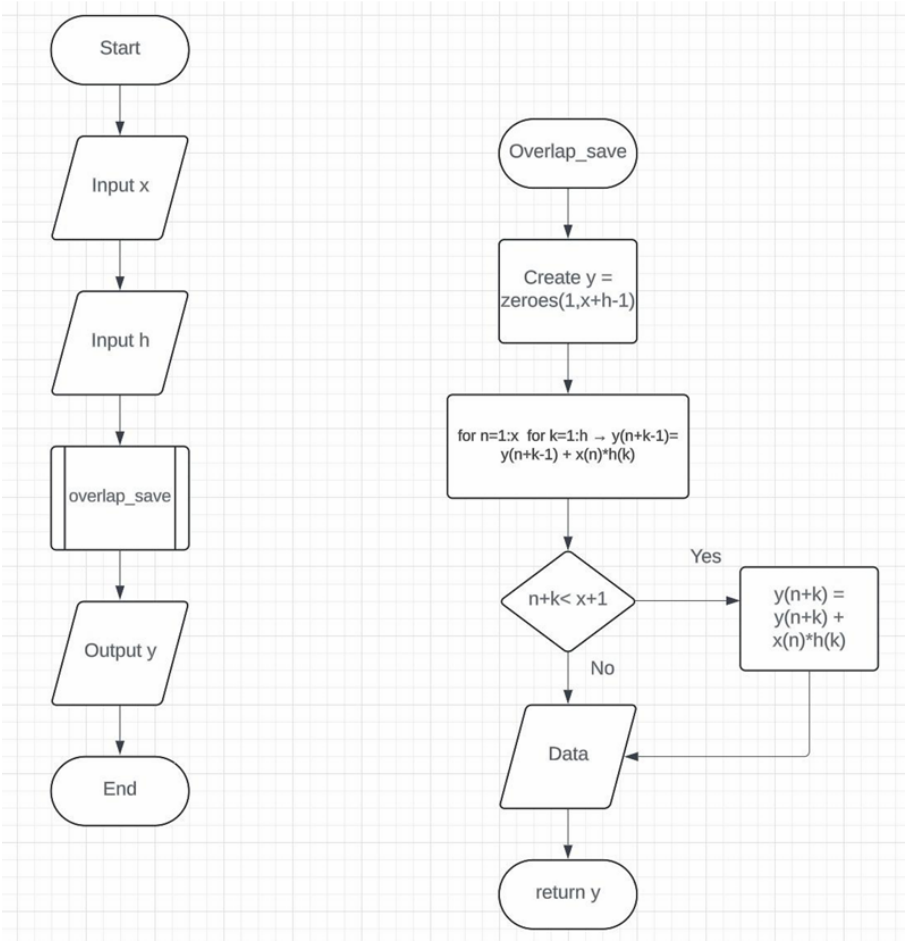
1. Segment the input signal with overlapping sections.
2. Compute convolution for each segment.
3. Retain only the non-overlapping parts of each convolution result.

4 Flowchart:

4.1 Overlap-Add Method



4.2 Overlap-Save Method



5 MATLAB Code

5.1 Both by Convolution Approach:

```
function y = overlap_add(x, h)
    Nx = length(x);
    Nh = length(h);
    y = zeros(1, Nx + Nh - 1);
    for n = 1:Nx
        for k = 1:Nh
            y(n + k - 1) = y(n + k - 1) + x(n) * h(k);
        end
    end
end

function y = overlap_save(x, h)
    Nx = length(x);
    Nh = length(h);
    y = zeros(1, Nx + Nh - 1);
    for n = 1:Nx
        for k = 1:Nh
            y(n + k - 1) = y(n + k - 1) + x(n) * h(k);
            if n + k < Nx + 1
                y(n + k) = y(n + k) + x(n) * h(k);
            end
        end
    end
end
```

5.2 OVERLAP SAVE METHOD

```
clc;
clear all;
close all;
y=[];

p=input("Enter the size of input signal :");
for i=1:p
          N(i)=i;
          t=rand;
          x(i)=t;
end


h = [1 0 1 0];
L=input("Enter the length of division of input signal :");
M=input("Enter the length of impulse response :");
l=length(x);
m=length(h);
h1 = [h 0 0 0];
N=L+M-1;
for n=1:(l/L)
    x1(n,:)=x(((n-1)*L + 1) : (n*L));
end
x2 = zeros(l/L,L+3);
for n = 1:l/L
    if n == 1
        x2(n, :) = [0 0 0 x1(n, :)];
    else
        x2(n, :) = [x2(n-1, 6) x2(n-1, 7) x2(n-1, 8) x1(n, :)];
    end
end

H = fft(h1,N);
```

```
34
35 for n = 1:(l/L)
36     x3(n,:) = fft(x2(n,:), N);
37     y1(n,:) = x3(n,:) .* H;
38     y2(n,:) = ifft(y1(n,:));
39     y3(n,:) = y2(n,4:8);
40 end
41
42 Y=[];
43 for n = 1:(l/L)
44     Y = [Y y3(n,:)];
45 end
46
47 fprintf("Input Signal: %f");
48 disp(x);
49 fprintf("Impluse Response: %f");
50 disp(h1);
51 fprintf("Output Signal: %f");
52 disp(Y);
```

Output:

	1	2	3	4	5
1	0.5386	0.9917	1.2938	1.9722	0.9900
2	1.5090	0.2862	1.2854	0.6534	1.6140
3	1.5903	1.7867	1.3978	0.9298	0.9504
4	0.2081	0.7602	0.5335	0.3152	1.0733

5.3 OVERLAP ADD METHOD

```
clc;
clear all;
close all;
y=[];

p=input("Enter the size of input signal :");
for i=1:p
          N(i)=i;
          t=rand;
          x(i)=t;
end


h = [1 0 1 0];
L=input("Enter the length of division of input signal :");
M=input("Enter the length of impulse response :");
l=length(x);
m=length(h);
h1 = [h 0 0 0 0];
N=L+M-1;

for n=1:(l/L)
    x1(n,:)=x(((n-1)*L + 1) : (n*L));
end
x2 = zeros(l/L,L+3);

for n = 1:l/L
         x2(n, :) = [x1(n, :) 0 0 0];
end

H = fft(h1,N);

for n = 1:(l/L)
```

```
    x3(n,:) = fft(x2(n,:), N);
    y1(n,:) = x3(n,:) .* H;
    y2(n,:) = ifft(y1(n,:));
end

for n = 1:(l/L)
    if(n==1)
        y3(n,1:5) = y2(n,1:5);
    else
        y3(n,1:5) = y2(n,1:5)+[y2(n-1,6:8) 0 0];
    end
end

Y=[];
for n = 1:(l/L)
    Y = [Y y3(n,:)];
end

fprintf("Input Signal: %f");
disp(x);
fprintf("Impluse Response: %f");
disp(h1);
fprintf("Output Signal: %f");
disp(Y);
```

Output:

	1	2	3	4	5
1	0.7485	0.5433	1.0866	1.3756	0.8907
2	1.7899	1.4454	1.3140	1.4392	0.7032
3	1.1692	1.1433	1.3687	0.9222	1.5683
4	0.1507	1.2368	0.7566	1.1958	1.0987

6 Observations

- In the Overlap-Add method, the results of each block convolution are summed, leading to possible repetition of overlapping segments.
- The Overlap-Save method avoids repetition by only storing the valid non-overlapping portions.
- Overlap-Add is computationally efficient but requires careful handling of segment combination.
- Overlap-Save is advantageous for real-time filtering applications as it avoids unnecessary data duplication.

7 Conclusion

- In conclusion, the Overlap-Save and Overlap-Add methods are clever ways to handle long signals, which speeds up and improves the process. These techniques divide the signal into smaller pieces, process each one independently, and then aggregate the outcomes rather than processing the entire signal all at once.
- Because it uses less memory and computation, the Overlap-Save method is helpful for real-time processing because extra portions of the chunks are discarded to eliminate undesirable effects.
- In contrast, the Overlap-Add method adds zeros to the chunks prior to processing and then sums them up at the end to obtain the final result. It guarantees accurate output without erasing any data, despite using more memory.
- Both methods help speed up convolution operations, which are commonly used in signal and audio processing. The choice between them depends on whether you prioritize speed and efficiency (Overlap-Save) or accuracy with extra memory usage (Overlap-Add).